Laboratory Record

submitted as part of

Programming for Data Science Lab (MACSE502)

LAB ASSESSMENT-10

by



Reg No: 26MML0031
Name: Hasnain H. Makada

To



Dr M Rajasekhara Babu

Dr M Rajasekhara Babu
School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
INDEX

Week No	Date	Sl. No.	Type	Topic	Title with download Link for document	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling	14
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. Normalize rating values.	18
3	23-07-2026	5	PP	NumPy	Creating and Manipulating Arrays Using NumPy	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape.	30
3	23-07-2026	6	EP	NumPy	Warehouse Shipment and Logistic Analytics Using NumPy	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape.	35
4	30-07-2026	7	PP	Pandas NumPy Matplotlib	Feature Engineering and Exploratory Data Analysis	Feature Engineering, Exploratory Data Analysis: Create a DataFrame with columns Feature1, Feature2, and Target containing random data. Perform the following operations: create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (low, medium, high), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 vs. Feature2 colored by Target, and interpret any visible patterns	46
4	30-07-2026	8	EP	Pandas NumPy Matplotlib	Employee Attrition Prediction Using Workforce Indicators	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing: OvertimeHours JobSatisfactionScore SalaryLevel TrainingHours WorkLifeBalance YearsAtCompany PromotionCount AttritionStatus Perform: 1. Create Log_OvertimeHours. 2. Categorize JobSatisfactionScore into: Disengaged, Satisfied, Highly Engaged. 3. Generate correlation matrix. 4. Create a Treemap Visualization of employee categories. 5. Analyze workforce trends.	54
5	06-08-2026	9	PP	Web Scraping	Web Scraping and Data Collection	Scrape data from a webpage and store it in a structured format like CSV orJSON	63
5	06-08-2026	10	EP	Web Scraping	Online Job Market Analytics	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.	70
6	20-08-2026	11	PP	Plotly	Interactive Bar	Create an interactive bar chart showing the population of	80

					Chart with Plotly	different cities	
6	20-08-2026	12	PP	Plotly	Interactive Scatter Plot with Plotly	Create an interactive scatter plot showing the relationship between house size and price	88
7	27-08-2026	13	PP	Bokeh	Interactive Scatter Plot with Bokeh	Create an interactive scatter plot showing the relationship between petal length and petal width from the Iris dataset using Bokeh	101
7	27-08-2026	14	EP	Bokeh	Interactive Line Plot with Bokeh	Create an interactive line plot showing the daily temperatures over a week using Bokeh.	105
7	27-08-2026	15	EP	Bokeh	Interactive Bar Chart with Bokeh	Create an interactive bar chart showing the sales figures of different products using Bokeh	109
8	03-09-2026	16	PP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly	113
8	03-09-2026	17	EP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly	122
9	10-09-2026	18	EP	arrays in Scala	arrays in Scala	Create an array of integers with the values 2, 5, 9, 14, 20. Write a function that takes this array and returns the sum of its elements. Next, create an array of strings with the names of five different fruits. Write a function that concatenates all elements of this array, separated by commas. Finally, iterate over the array of integers and print each element to the console	131
10	17-09-2026	19	EP	lists, sets, and tuples in Scala	lists, sets, and tuples in Scala	Create a list of the first five prime numbers. Write a function that takes this list and returns a new list with each element squared. Then, create a set of unique characters from the string "hello world" and write a function that takes two sets and returns their intersection. Create a tuple with three elements: an integer, a string, and a boolean, then access and print each element	134
11	24-09-2026	20	EP	HashMap’s	Handling collisions and resolving conflicts in HashMap’s	Create a HashMap with keys representing student names and values representing their grades, then insert multiple entries including a duplicate key with a different grade. Write a function to merge two HashMap’s, resolving conflicts by taking the higher grade. Finally, write a function to handle collisions by chaining, using lists to store multiple values for a single key.	137
12	08-10-2026	21	EP	Exploring advanced functional combinators beyond the basic set	Exploring advanced functional combinators beyond the basic set	Create a list of integers from 1 to 10, then use filter to create a new list with only even numbers. Use map to create a new list where each element is multiplied by 3. Use flatMap to create a list of tuples where each integer is paired with its square. Finally, use foldLeft to calculate the product of all elements in the list.	140
13	15-10-2026	22	EP	Case Classes	Case Classes	Create a case class Person with fields name, age, and city. Create a list of Person objects representing five different individuals. Write a function to filter out people older than 30. Next, write a function that groups people by their city. Then, write a function that transforms each Person object into a string in the format "Name is Age years old and lives in City". Finally, write a function to sort the list of people by age in ascending order	143
13	15-10-2026	23	EP	Use Saddle to load and manipulate a dataset	Use Saddle to load and manipulate a dataset	Use Saddle to load and manipulate a dataset of fruit prices and quantities, filtering for apples and calculating their average price. You will then visualize the price trend over time using Vegas. This exercise covers basic data manipulation with Saddle and interactive visualization with Vegas.	146
14	22-10-2026	24	EP	Use Breeze library	Use Breeze library	Use Breeze library to calculate total revenue for each fruit type by multiplying price and quantity from a dataset. You will then create a bar chart to visualize these revenues using Plotly. Scala. This exercise introduces numerical computations with Breeze and dynamic visualizations with Plotly. Scala.	150
14	22-10-2026	25	EP	Saddle for data manipulation	Saddle for data manipulation	Analyse fruit prices using Spire for precise statistics and Saddle for data manipulation. You will compute the mean and variance of prices for each fruit and visualize these trends over time using Vegas. This exercise demonstrates the use of Spire for numerical precision, advanced data manipulation with Saddle, and effective visualization with Vegas	153

PP: Practice Program; EP: Exercise Program; CP: Challenging Program

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 10/09/2026 **Week No.: 9** **Program S. No. 18**

Title	Arrays in Scala
Problem Statement	Create an array of integers with the values 2, 5, 9, 14, 20. Write a function that takes this array and returns the sum of its elements. Next, create an array of strings with the names of five different fruits. Write a function that concatenates all elements of this array, separated by commas. Finally, iterate over the array of integers and print each element to the console.
Objectives	• To declare and initialize fixed-size arrays in Scala. • To write a user-defined function that aggregates an integer array. • To write a user-defined function that joins a string array. • To traverse an array with an index loop and print every element. • To verify a computed aggregate against an independent calculation.
Dataset URL Link / File	No external dataset is required. The inputs are declared inside the program: the integer array (2, 5, 9, 14, 20) and the string array (Apple, Banana, Mango, Grapes, Orange).

Functions Details

Sl. No.	Function	Package	Description of function
1	`Array()`	Scala	Creates a fixed-size array from the supplied elements.
2	`sum`	Scala	Returns the total of all elements of a numeric array.
3	`mkString()`	Scala	Joins every element into one string using a separator.
4	`length`	Scala	Returns the number of elements stored in the array.
5	`apply(index)`	Scala	Returns the element stored at the given index.
6	`Array[Int]`	Scala	Declares an array whose elements are integers.
7	`Array[String]`	Scala	Declares an array whose elements are strings.
8	`def`	Scala	Declares the user-defined functions used by the program.
9	`while`	Scala	Repeats the traversal while the index is below the length.
10	`println()`	Scala	Writes one formatted line of text to the console.
11	`==` `(equality)`	Scala	Compares the manual sum with the function result.
12	`main()`	Scala	Entry point that executes the program.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Declare the integer array with the values 2, 5, 9, 14 and 20.
2. Define sumOfArray() that returns the total of the array elements.
3. Print the integer array, its length and the computed sum.
4. Declare the string array containing the names of five fruits.
5. Define concatenateStrings() that joins the elements using mkString.
6. Define printElements() that walks the array with an index loop.
7. Call printElements() so every integer is printed with its index.
8. Compute the sum manually and compare it with the function result.
9. Print the verification line and terminate the program.

Program

```
// Program 18 - Arrays in Scala
// Week 9 | 10-09-2026 | Exercise Program
object Program18 {

  // (a) Function that returns the sum of all elements of an integer array
  def sumOfArray(numbers: Array[Int]): Int = numbers.sum

  // (b) Function that concatenates all elements of a string array, separated by commas
  def concatenateStrings(names: Array[String]): String = names.mkString(", ")

  // (c) Function that prints every element of an integer array
```

Student Reg. No: 26MML0031 Name: Hasnain Makada
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

Date: 10/09/2026 **Week No.: 9** **Program S. No. 18**

```
  def printElements(numbers: Array[Int]): Unit = {
    var index = 0
    while (index < numbers.length) {
      println("  Element at index " + index + " = " + numbers(index))
      index += 1
    }
  }

  def main(args: Array[String]): Unit = {
    println("=" * 62)
    println("PROGRAM 18 : ARRAYS IN SCALA")
    println("=" * 62)

    val numbers = Array(2, 5, 9, 14, 20)
    println("\n1. Integer array         : " + numbers.mkString("[", ", ", "]"))
    println("   Length                : " + numbers.length)
    println("   Sum of all elements   : " + sumOfArray(numbers))

    val fruits = Array("Apple", "Banana", "Mango", "Grapes", "Orange")
    println("\n2. String array (fruits) : " + fruits.mkString("[", ", ", "]"))
    println("   Concatenated           : " + concatenateStrings(fruits))

    println("\n3. Iterating over the integer array:")
    printElements(numbers)

    val manualSum = 2 + 5 + 9 + 14 + 20
    println("\n4. Verification           : manual sum = " + manualSum +
            " , function sum = " + sumOfArray(numbers) +
            " , match = " + (manualSum == sumOfArray(numbers)))
    println("=" * 62)
  }
}
```

Output Snapshots With Explanation/ Observations

```
Program18.scala

hasnain@vit:~/WEEK 9/programs$
scala Program18
==============================================================
PROGRAM 18 : ARRAYS IN SCALA
==============================================================

1. Integer array         : [2, 5, 9, 14, 20]
   Length                : 5
   Sum of all elements   : 50
```

Fig. 1.1: Terminal execution showing creation of the integer array, its length and the sum returned by the user-defined function. The string array is concatenated using mkString().

The integer array is printed first with its length, and the user-defined function returns the sum 50, which the next line confirms against the manually calculated value. The string array is joined with mkString so the five fruit names appear as one comma-separated line.

```
Program18.scala

hasnain@vit:~/WEEK 9/programs$
scala Program18
2. String array (fruits) : [Apple, Banana, Mango, Grapes, Orange]
   Concatenated          : Apple, Banana, Mango, Grapes, Orange

3. Iterating over the integer array:
  Element at index 0 = 2
  Element at index 1 = 5
  Element at index 2 = 9
  Element at index 3 = 14
  Element at index 4 = 20

4. Verification          : manual sum = 50 , function sum = 50 , match = true
==============================================================
```

Fig. 1.2: Index-based traversal printing every element of the integer array, followed by the verification line confirming that the function result matches the manual sum.

The index loop walks the array from position 0 to 4 and prints each element beside its index, which shows that a Scala array is addressable by position. The final verification line reports that the manual sum and the function result are equal.

Conclusions

The integer array (2, 5, 9, 14, 20) was created successfully and the user-defined function returned a total of 50, which matched the manually calculated value. The string array of five fruits was concatenated into a single comma-separated string using mkString(). The index-based traversal printed every element of the integer array together with its position. The experiment demonstrated that Scala arrays are fixed-size, index-addressable collections and that aggregation and string joining can be expressed concisely through built-in collection methods.

References

1. Scala Array documentation:
https://www.scala-lang.org/api/current/scala/Array.html

	2. Scala Collections overview: https://docs.scala-lang.org/overviews/collections/overview.html 3. Class lecture: Arrays and array operations in Scala. 4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%209/programs/Program18_Arrays.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 17/09/2026 **Week No.: 10** **Program S. No. 19**

Title	**Lists, Sets and Tuples in Scala**
Problem Statement	Create a list of the first five prime numbers. Write a function that takes this list and returns a new list with each element squared. Then, create a set of unique characters from the string hello world and write a function that takes two sets and returns their intersection. Create a tuple with three elements: an integer, a string and a boolean, then access and print each element.
Objectives	• To create immutable lists and transform their elements with a function. • To build a set of unique characters from a string. • To compute the intersection of two sets. • To construct a tuple holding three different types and access its components. • To compare immutable collections with the arrays used earlier.
Dataset URL Link / File	No external dataset is required. The inputs are the first five prime numbers (2, 3, 5, 7, 11), the literal string hello world and the tuple (101, Hasnain Makada, true).

Functions Details

Sl. No.	Function	Package	Description of function
1	`List()`	Scala	Creates an immutable linked list of elements.
2	`map()`	Scala	Applies a function to every element, returning a new list.
3	`sum`	Scala	Returns the total of all elements of a numeric list.
4	`toSet`	Scala	Converts a string or collection into a set of unique values.
5	`Set()`	Scala	Creates an immutable set that rejects duplicates.
6	`intersect()`	Scala	Returns the elements common to two sets.
7	`size`	Scala	Returns the number of elements in a set or list.
8	`toList`	Scala	Converts a set into a list so that it can be sorted.
9	`sorted`	Scala	Orders the elements of a collection in ascending order.
10	`Tuple3 / ( )`	Scala	Creates a tuple grouping three values of mixed types.
11	`_1, _2, _3`	Scala	Accesses the first, second and third tuple components.
12	`println()`	Scala	Writes one formatted line of text to the console.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Create the list of the first five prime numbers 2, 3, 5, 7 and 11.
2. Define squareList() that maps every element of the list to its square.
3. Print the original list, the squared list and the sum of the squares.
4. Convert the string hello world into a set of unique characters.
5. Print the unique characters and their count.
6. Define intersectSets() that returns the common elements of two sets.
7. Intersect the character set with the vowel set and print the result.
8. Create the tuple (101, Hasnain Makada, true).
9. Access and print each tuple component using _1, _2 and _3.

Program

```
// Program 19 - Lists, Sets and Tuples in Scala
// Week 10 | 17-09-2026 | Exercise Program
object Program19 {

  // (a) Function that squares every element of a list
  def squareList(numbers: List[Int]): List[Int] = numbers.map(n => n * n)

  // (b) Function that returns the intersection of two sets
  def intersectSets(first: Set[Char], second: Set[Char]): Set[Char] =
first.intersect(second)

  // (c) Function that prints each element of a tuple
  def printTuple(record: (Int, String, Boolean)): Unit = {
    println("  First  element (Int)     : " + record._1)
    println("  Second element (String)  : " + record._2)
    println("  Third  element (Boolean) : " + record._3)
  }
```

```
  def main(args: Array[String]): Unit = {
    println("=" * 62)
    println("PROGRAM 19 : LISTS, SETS AND TUPLES IN SCALA")
    println("=" * 62)

    val primes = List(2, 3, 5, 7, 11)
    println("\n1. List of first five primes : " + primes)
    println("   Squares of each element    : " + squareList(primes))
    println("   Sum of squares             : " + squareList(primes).sum)

    val text = "hello world"
    val uniqueChars = text.toSet
    println("\n2. Source string             : " + text)
    println("   Unique characters (Set)    : " + uniqueChars.toList.sorted.mkString("[", ",
", "]"))
    println("   Number of unique chars     : " + uniqueChars.size)

    val vowels = Set('a', 'e', 'i', 'o', 'u')
    println("   Vowel set                  : " + vowels.toList.sorted.mkString("[", ", ",
"]"))
    println("   Intersection (vowels)      : " + intersectSets(uniqueChars,
vowels).toList.sorted.mkString("[", ", ", "]"))

    val record = (101, "Hasnain Makada", true)
    println("\n3. Tuple                      : " + record)
    printTuple(record)
    println("=" * 62)
  }
}
```

Output Snapshots With Explanation/ Observations

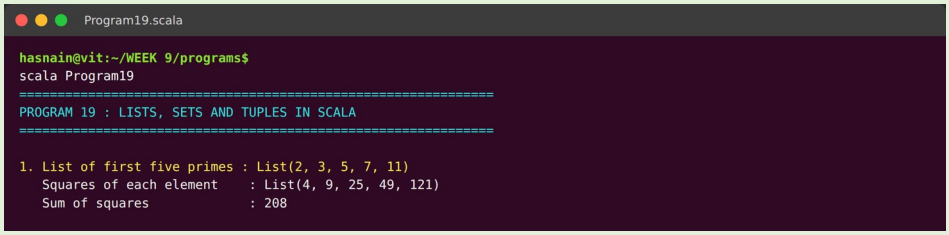


Fig. 2.1: Terminal execution showing the list of prime numbers, the squared list produced by the user-defined function and the sum of the squares.

The list of five primes is printed, followed by the squared list produced by the mapping function and the sum of those squares, so the transformation applied to every element is visible in one place.

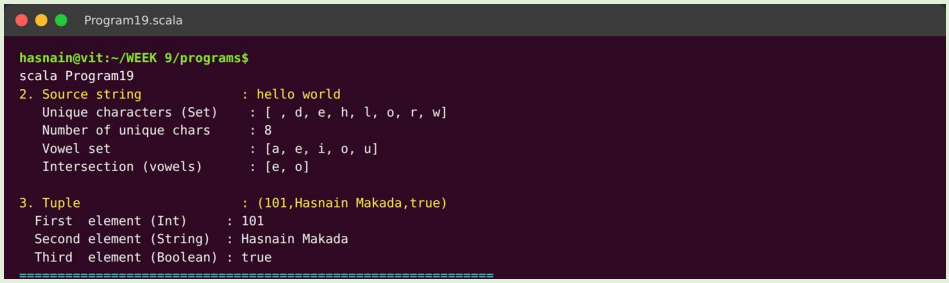


Fig. 2.2: Unique characters extracted from the string, the vowel intersection and the three components of the tuple printed individually.

Converting the string to a set removes the repeated letters and leaves eight unique characters. Intersecting that set with the vowel set returns only e and o, and the tuple is then printed whole before each of its three components is echoed individually.

Conclusions

A list of the first five prime numbers was created and the mapping function returned the squared list (4, 9, 25, 49, 121) with a total of 208. Converting the string hello world into a set removed all duplicate letters and left eight unique characters, and the intersection with the vowel set produced {e, o}. The three-element tuple was created and each component was accessed and printed individually. The experiment showed that Scala lists preserve order, sets guarantee uniqueness and tuples group heterogeneous values without a dedicated class.

References

1. Scala List documentation: https://www.scala-lang.org/api/current/scala/collection/immutable/List.html
2. Scala Set documentation: https://www.scala-lang.org/api/current/scala/collection/immutable/Set.html
3. Scala Tuples guide: https://docs.scala-lang.org/tour/tuples.html

	4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%209/programs/Program19_ListsSetsTuples.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 24/09/2026 **Week No.: 11** **Program S. No. 20**

Title	HashMaps: Handling Collisions and Resolving Conflicts
Problem Statement	Create a HashMap with keys representing student names and values representing their grades, then insert multiple entries including a duplicate key with a different grade. Write a function to merge two HashMaps, resolving conflicts by taking the higher grade. Finally, write a function to handle collisions by chaining, using lists to store multiple values for a single key.
Objectives	• To create and update a mutable HashMap of student names and grades. • To observe how a duplicate key overwrites the previously stored value. • To merge two HashMaps while resolving key conflicts deterministically. • To implement separate chaining so colliding keys share a bucket. • To explain the role of the hash function in distributing keys.
Dataset URL Link / File	No external dataset is required. All records are declared inside the program: student names paired with integer grades, plus a second set of six names used to force collisions across four buckets.

Functions Details

Sl. No.	Function	Package	Description of function
1	`mutable.HashMap()`	Scala	Creates a mutable hash table keyed by student name.
2	`+= (key -> value)`	Scala	Inserts a new entry or overwrites an existing key.
3	`get()`	Scala	Returns the value for a key wrapped in an Option.
4	`match / Some / None`	Scala	Selects the conflict-resolution branch for a key.
5	`++= (collection)`	Scala	Copies all entries of one map into another.
6	`toSeq.sorted`	Scala	Orders entries so the map prints deterministically.
7	`hashCode`	Scala	Produces the integer hash used to choose a bucket.
8	`abs / % (modulus)`	Scala	Maps the hash onto a valid bucket index.
9	`Array.fill()`	Scala	Creates the fixed array of bucket chains.
10	`List / :: (cons)`	Scala	Stores several colliding entries in one bucket chain.
11	`collect()`	Scala	Extracts all values that match a key inside a chain.
12	`foreach / for`	Scala	Traverses the bucket array to print occupied buckets.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Create a mutable HashMap with three student names and their grades. 2. Insert two more students, then insert an existing key with a different grade. 3. Observe that the duplicate key replaces the earlier grade. 4. Define mergeTakingHigher() that copies the first map into a new map. 5. For every key of the second map keep the larger of the two grades. 6. Print both input maps and the merged map to show resolved conflicts. 7. Define a ChainedHashMap class with a fixed number of buckets. 8. Compute the bucket index as |key.hashCode| % bucketCount. 9. Insert six names so several of them land in the same bucket. 10. Print the non-empty buckets to show the chains built by separate chaining.

Program	`// Program 20 - HashMaps: collisions and conflict resolution`
	`// Week 11 | 24-09-2026 | Exercise Program`
	`import scala.collection.mutable`

Student Reg. No: 26MML0031 Name: Hasnain Makada
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

```
// A HashMap that resolves collisions by separate chaining with lists
class ChainedHashMap(bucketCount: Int) {

  private val table: Array[List[(String, Int)]] = Array.fill(bucketCount)
(List.empty[(String, Int)])

  // Hash function that maps a key to a bucket index
  private def bucketIndex(key: String): Int = math.abs(key.hashCode) % bucketCount

  // Insert a key-value pair, prepending it to the bucket chain
  def put(key: String, grade: Int): Unit = {
    val index = bucketIndex(key)
    table(index) = (key, grade) :: table(index)
  }

  // Look up every value stored under a key (a chain may hold several)
  def get(key: String): List[Int] =
    table(bucketIndex(key)).collect { case (k, v) if k == key => v }

  // Show the non-empty buckets, proving that chaining holds the collisions
  def display(): Unit = {
    println("   Bucket | Chain (key -> grade)")
    println("   -------+----------------------")
    for (index <- table.indices; if table(index).nonEmpty) {
      val chain = table(index).reverse.map { case (k, v) => k + " -> " + v }.mkString(" , ")
      println("   " + index + "      | " + chain)
    }
  }
}

object Program20 {

  // (a) Merge two HashMaps, resolving conflicts by keeping the higher grade
  def mergeTakingHigher(first: mutable.HashMap[String, Int],
                        second: mutable.HashMap[String, Int]): mutable.HashMap[String, Int]
= {
    val merged = mutable.HashMap[String, Int]()
    merged ++= first
    for ((name, grade) <- second) {
      merged.get(name) match {
        case Some(existing) => if (grade > existing) merged(name) = grade
        case None           => merged(name) = grade
      }
    }
    merged
  }

  // (b) Handle collisions by chaining when many keys land in one bucket
  def demonstrateChaining(): Unit = {
    val chained = new ChainedHashMap(4)
    val entries = List("Amit" -> 78, "Neha" -> 85, "Rahul" -> 66, "Sara" -> 92,
                       "Vikram" -> 74, "Zoya" -> 88)
    for ((name, grade) <- entries) chained.put(name, grade)
    println("   Hash function      : index = |key.hashCode| % 4")
    for ((name, _) <- entries) {
      println("   " + name + " -> bucket " + (math.abs(name.hashCode) % 4))
    }
    println()
    chained.display()
  }

  def main(args: Array[String]): Unit = {
    println("=" * 62)
    println("PROGRAM 20 : HASHMAPS - COLLISIONS AND CONFLICT RESOLUTION")
    println("=" * 62)

    // 1. Create a HashMap of student names and grades
    val grades = mutable.HashMap(
      "Hasnain" -> 88,
      "Amit"    -> 76,
      "Neha"    -> 91
    )
    println("\n1. Initial HashMap           : " + grades.toSeq.sorted.mkString("{", ", ",
"}"))

    // 2. Insert multiple entries, including a duplicate key with a different grade
    grades += ("Rahul" -> 69)
    grades += ("Sara"  -> 95)
    grades += ("Hasnain" -> 93)
    println("   After inserts + duplicate : " + grades.toSeq.sorted.mkString("{", ", ",
"}"))
    println("   Note: the duplicate key Hasnain replaced 88 with " + grades("Hasnain"))

    // 3. Merge two HashMaps, resolving conflicts by taking the higher grade
    val sectionA = mutable.HashMap("Hasnain" -> 93, "Amit" -> 76, "Zoya" -> 81)
    val sectionB = mutable.HashMap("Hasnain" -> 85, "Zoya" -> 90, "Vikram" -> 74)
    val merged   = mergeTakingHigher(sectionA, sectionB)
    println("\n2. Section A                 : " + sectionA.toSeq.sorted.mkString("{", ", ",
"}"))
```

```
    println("   Section B                 : " + sectionB.toSeq.sorted.mkString("{", ", ",
"}"))
    println("   Merged (higher grade wins): " + merged.toSeq.sorted.mkString("{", ", ",
"}"))
    println("   Hasnain 93 vs 85 -> " + merged("Hasnain") + " | Zoya 81 vs 90 -> " +
merged("Zoya"))

    // 4. Collision handling by chaining
    println("\n3. Collision handling by separate chaining:")
    demonstrateChaining()
    println("=" * 62)
  }
}
```

Output Snapshots With Explanation/ Observations

```
Program20.scala

hasnain@vit:~/WEEK 9/programs$
scala Program20
==============================================================
PROGRAM 20 : HASHMAPS - COLLISIONS AND CONFLICT RESOLUTION
==============================================================

1. Initial HashMap          : {(Amit,76), (Hasnain,88), (Neha,91)}
   After inserts + duplicate : {(Amit,76), (Hasnain,93), (Neha,91), (Rahul,69), (Sara,95)}
   Note: the duplicate key Hasnain replaced 88 with 93
```

Fig. 3.1: Terminal execution showing the initial HashMap, insertion of new entries, the duplicate-key overwrite and the merge of two HashMaps taking the higher grade.

The initial map holds three students. After the inserts the duplicate key Hasnain replaces the earlier grade of 88 with 93, which shows that a HashMap keeps one value per key. The merged map then resolves the overlapping names by keeping the higher grade.

```
Program20.scala

hasnain@vit:~/WEEK 9/programs$
scala Program20
2. Section A                : {(Amit,76), (Hasnain,93), (Zoya,81)}
   Section B                : {(Hasnain,85), (Vikram,74), (Zoya,90)}
   Merged (higher grade wins): {(Amit,76), (Hasnain,93), (Vikram,74), (Zoya,90)}
   Hasnain 93 vs 85 -> 93 | Zoya 81 vs 90 -> 90

3. Collision handling by separate chaining:
   Hash function      : index = |key.hashCode| % 4
   Amit -> bucket 3
   Neha -> bucket 0
   Rahul -> bucket 0
   Sara -> bucket 1
   Vikram -> bucket 2
   Zoya -> bucket 1

   Bucket | Chain (key -> grade)
   -------+--------------------
   0      | Neha -> 85 , Rahul -> 66
   1      | Sara -> 92 , Zoya -> 88
   2      | Vikram -> 74
   3      | Amit -> 78
==============================================================
```

Fig. 3.2: Bucket assignment produced by the hash function and the non-empty buckets showing separate chaining for colliding keys.

The hash function assigns every name to one of four buckets, and two pairs of names land in the same bucket. The bucket table printed underneath lists both entries in each of those chains, so no record is lost even though two keys share an index.

Conclusions

The initial HashMap stored three students and the duplicate insertion for Hasnain replaced the earlier grade of 88 with 93, confirming that a HashMap allows only one value per key. Merging two HashMaps with the higher-grade rule produced a combined map in which Hasnain kept 93 and Zoya was upgraded from 81 to 90. The chained implementation placed six keys into four buckets, with bucket 0 holding Neha and Rahul and bucket 1 holding Sara and Zoya, so no entry was lost even though the hash function assigned two keys to the same index.

References

1. Scala HashMap documentation: https://www.scala-lang.org/api/current/scala/collection/mutable/HashMap.html
2. Hash table collision resolution: https://en.wikipedia.org/wiki/Hash_table#Collision_resolution
3. Class lecture: Maps and HashMaps in Scala.
4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%209/programs/Program20_HashMapCollisions.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 08/10/2026 **Week No.: 12** **Program S. No. 21**

Title	Advanced Functional Combinators in Scala
Problem Statement	Create a list of integers from 1 to 10, then use filter to create a new list with only even numbers. Use map to create a new list where each element is multiplied by 3. Use flatMap to create a list of tuples where each integer is paired with its square. Finally, use foldLeft to calculate the product of all elements in the list.
Objectives	• To apply filter, map and flatMap to an immutable list of integers. • To distinguish flatMap from map by the shape of the returned collection. • To reduce a collection to a single value with foldLeft. • To chain several combinators into one readable transformation pipeline. • To compare each combinator result against known values.
Dataset URL Link / File	No external dataset is required. The source collection is the list of integers from 1 to 10 generated with (1 to 10).toList.

Functions Details

Sl. No.	Function	Package	Description of function
1	`(1 to 10).toList`	Scala	Builds the source list of ten integers.
2	`filter()`	Scala	Keeps only the elements that satisfy a predicate.
3	`map()`	Scala	Transforms every element, preserving the collection length.
4	`flatMap()`	Scala	Transforms each element and flattens results into one list.
5	`foldLeft()`	Scala	Reduces the list left to right using an accumulator.
6	`product`	Scala	Multiplies all elements, used to verify foldLeft.
7	`sum`	Scala	Adds all elements, used to total the mapped list.
8	`length`	Scala	Counts the elements produced by a combinator.
9	`mkString()`	Scala	Joins the resulting elements into one display string.
10	`List((n, n*n))`	Scala	Wraps each pair so flatMap can flatten it.
11	`println()`	Scala	Writes one formatted line of text to the console.
12	`main()`	Scala	Entry point that executes the program.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Build the source list of the integers 1 to 10. 2. Apply filter with the predicate n % 2 == 0 to keep the even numbers. 3. Apply map with the function n * 3 to triple every element. 4. Apply flatMap so each integer is paired with its square. 5. Apply foldLeft with seed 1 and multiplication to obtain 10 factorial. 6. Verify the foldLeft result against the built-in product of the list. 7. Chain filter, map and foldLeft into a single transformation pipeline. 8. Print a summary of how many elements each combinator produced.
Program	

```
// Program 21 - Advanced functional combinators
// Week 12 | 08-10-2026 | Exercise Program
object Program21 {

  def main(args: Array[String]): Unit = {
    println("=" * 62)
    println("PROGRAM 21 : ADVANCED FUNCTIONAL COMBINATORS")
    println("=" * 62)

    val numbers = (1 to 10).toList
    println("\n1. Source list (1 to 10)     : " + numbers.mkString("[", ", ", "]"))

    // 2. filter - keep only the even numbers
    val evens = numbers.filter(n => n % 2 == 0)
    println("\n2. filter(n % 2 == 0)        : " + evens.mkString("[", ", ", "]"))
    println("   Count of even numbers     : " + evens.length)

    // 3. map - multiply every element by 3
    val tripled = numbers.map(n => n * 3)
    println("\n3. map(n => n * 3)           : " + tripled.mkString("[", ", ", "]"))
```

Student Reg. No: 26MML0031 Name: Hasnain Makada
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

```
    println("   Sum after mapping         : " + tripled.sum)

    // 4. flatMap - pair every integer with its square
    val pairs = numbers.flatMap(n => List((n, n * n)))
    println("\n4. flatMap(n => (n, n*n))    :")
    println("   " + pairs.mkString("[", ", ", "]"))
    println("   Number of pairs produced  : " + pairs.length)

    // 5. foldLeft - product of all elements
    val product = numbers.foldLeft(1)((accumulator, n) => accumulator * n)
    println("\n5. foldLeft(1)(_ * _)        : " + product)
    println("   Verification (10!)         : " + (1 to 10).product +
            " , match = " + (product == (1 to 10).product))

    // 6. Combined pipeline using several combinators together
    val pipeline = numbers
      .filter(n => n % 2 == 0)
      .map(n => n * 3)
      .foldLeft(0)((accumulator, n) => accumulator + n)
    println("\n6. Combined pipeline")
    println("   evens -> x3 -> sum        : " + pipeline)

    println("\n7. Summary of combinator results")
    println("   filter   -> " + evens.length + " elements")
    println("   map      -> " + tripled.length + " elements")
    println("   flatMap  -> " + pairs.length + " elements")
    println("   foldLeft -> single value " + product)
    println("=" * 62)
  }
}
```

Output Snapshots With Explanation/ Observations

```
Program21.scala

hasnain@vit:~/WEEK 9/programs$
scala Program21
==============================================================
PROGRAM 21 : ADVANCED FUNCTIONAL COMBINATORS
==============================================================

1. Source list (1 to 10)     : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

2. filter(n % 2 == 0)        : [2, 4, 6, 8, 10]
   Count of even numbers     : 5
```

Fig. 4.1: Terminal execution showing the source list, the filtered even numbers, the tripled list and the flatMap pairs generated from the input list.

The source list 1 to 10 is shown, followed by the five even numbers returned by filter and the ten tripled values returned by map. The flatMap block pairs every integer with its square and flattens the pairs into a single list.

```
Program21.scala

hasnain@vit:~/WEEK 9/programs$
scala Program21
3. map(n => n * 3)           : [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
   Sum after mapping         : 165

4. flatMap(n => (n, n*n))    :
   [(1,1), (2,4), (3,9), (4,16), (5,25), (6,36), (7,49), (8,64), (9,81), (10,100)]
   Number of pairs produced  : 10

5. foldLeft(1)(_ * _)        : 3628800
   Verification (10!)         : 3628800 , match = true

6. Combined pipeline
   evens -> x3 -> sum        : 90

7. Summary of combinator results
   filter   -> 5 elements
   map      -> 10 elements
   flatMap  -> 10 elements
   foldLeft -> single value 3628800
==============================================================
```

Fig. 4.2: foldLeft product with its verification against the built-in product, the combined pipeline result and the summary of combinator outputs.

foldLeft reduces the list to 3628800, and the verification line confirms that this equals the built-in product of 1 to 10. The combined pipeline then applies filter, map and foldLeft in one expression to give 90, and the summary counts how many elements each combinator produced.

Conclusions

Applying filter to the list 1 to 10 returned the five even numbers (2, 4, 6, 8, 10), while map with multiplication by three produced a list of the same length containing (3, 6, 9, ..., 30). flatMap paired every integer with its square and flattened the pairs into a single list of ten tuples, and foldLeft returned 3628800, which matches the built-in product and confirms 10 factorial. The combined pipeline of filter, map and foldLeft produced 90 in a single expression, showing that functional combinators can replace

	explicit loops with shorter and more declarative code.
References	1. Scala collections operations: https://docs.scala-lang.org/overviews/collections/operations.html 2. Scala tour: anonymous function syntax: https://docs.scala-lang.org/tour/anonymous-function-syntax.html 3. Class lecture: Functional combinators in Scala. 4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%209/programs/Program21_FunctionalCombinators.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 15/10/2026 **Week No.: 13** **Program S. No. 22**

Title	Case Classes in Scala
Problem Statement	Create a case class Person with fields name, age, and city. Create a list of Person objects representing five different individuals. Write a function to filter out people older than 30. Next, write a function that groups people by their city. Then, write a function that transforms each Person object into a string in the format "Name is Age years old and lives in City". Finally, write a function to sort the list of people by age in ascending order.
Objectives	• To declare a case class and construct instances with the automatic apply method. • To filter a list of case-class objects with a predicate function. • To group objects by one of their fields using groupBy. • To transform objects into formatted strings with map. • To sort a collection of objects by a numeric field and to use the copy, equals, toString and productArity members that a case class provides.
Dataset URL Link / File	No external dataset is required. Five Person records are declared inside the program: Hasnain (22, Delhi), Amit (35, Mumbai), Neha (41, Delhi), Rahul (28, Pune) and Sara (33, Mumbai).

Functions Details

Sl. No.	Function	Package	Description of function
1	`case class`	Scala	Declares a class with apply, equals, hashCode, toString and copy.
2	`List()`	Scala	Creates an immutable list of Person objects.
3	`filter()`	Scala	Keeps only the elements that satisfy a predicate.
4	`groupBy()`	Scala	Partitions the list into a Map keyed by the given field.
5	`map()`	Scala	Transforms every Person into a formatted description.
6	`sortBy()`	Scala	Orders the list by the supplied numeric field.
7	`copy()`	Scala	Produces a modified copy of a case-class instance.
8	`equals()` / `==`	Scala	Compares two case-class instances by value.
9	`toString()`	Scala	Renders a case-class instance in a readable form.
10	`productArity`	Scala	Returns the number of fields held by the case class.
11	`mkString()`	Scala	Joins the names of a grouped list into one string.
12	`println()`	Scala	Writes one formatted line of text to the console.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Declare case class Person with the fields name, age and city.
2. Build a list of five Person objects.
3. Define olderThan30() using filter to keep the people older than 30.
4. Print the filtered people and their count.
5. Define groupByCity() using groupBy on the city field.
6. Print each city with the names of the people who live there.
7. Define describe() to map each Person to the required sentence.
8. Print every formatted description.
9. Define sortByAge() using sortBy on the age field and print the sorted list.
10. Demonstrate copy, equals, toString and productArity on one instance.

Program

```
// Program 22 - Case Classes
// Week 13 | 15-10-2026 | Exercise Program
object Program22 {

  // The case class required by the problem statement
  case class Person(name: String, age: Int, city: String)

  // (a) Keep only the people older than 30
  def olderThan30(people: List[Person]): List[Person] =
    people.filter(p => p.age > 30)
```

```
  // (b) Group the people by the city they live in
  def groupByCity(people: List[Person]): Map[String, List[Person]] =
    people.groupBy(p => p.city)

  // (c) Render each person as "Name is Age years old and lives in City"
  def describe(people: List[Person]): List[String] =
    people.map(p => p.name + " is " + p.age + " years old and lives in " + p.city)

  // (d) Sort the people by age, youngest first
  def sortByAge(people: List[Person]): List[Person] =
    people.sortBy(p => p.age)

  def main(args: Array[String]): Unit = {
    println("=" * 68)
    println("PROGRAM 22 : CASE CLASSES IN SCALA")
    println("=" * 68)

    val people = List(
      Person("Hasnain", 22, "Delhi"),
      Person("Amit", 35, "Mumbai"),
      Person("Neha", 41, "Delhi"),
      Person("Rahul", 28, "Pune"),
      Person("Sara", 33, "Mumbai")
    )

    println("\n1. List of Person objects")
    for (p <- people) println("   " + p)

    println("\n2. People older than 30")
    val older = olderThan30(people)
    for (p <- older) println("   " + p.name + " (" + p.age + ")")
    println("   Count: " + older.length + " of " + people.length)

    println("\n3. Grouped by city")
    val grouped = groupByCity(people)
    for (city <- grouped.keys.toList.sorted) {
      println("   " + city + " -> " + grouped(city).map(_.name).mkString(", "))
    }

    println("\n4. Formatted descriptions")
    for (line <- describe(people)) println("   " + line)

    println("\n5. Sorted by age (ascending)")
    val sorted = sortByAge(people)
    for (p <- sorted) println("   " + p.age + "  " + p.name + "  (" + p.city + ")")

    println("\n6. Case class features demonstrated")
    val a = Person("Hasnain", 22, "Delhi")
    val b = Person("Hasnain", 22, "Delhi")
    println("   copy():      " + a.copy(age = 23))
    println("   equals():    " + (a == b))
    println("   toString():  " + a.toString)
    println("   productArity:" + a.productArity)
    println("=" * 68)
  }
}
```

Output Snapshots With Explanation/ Observations

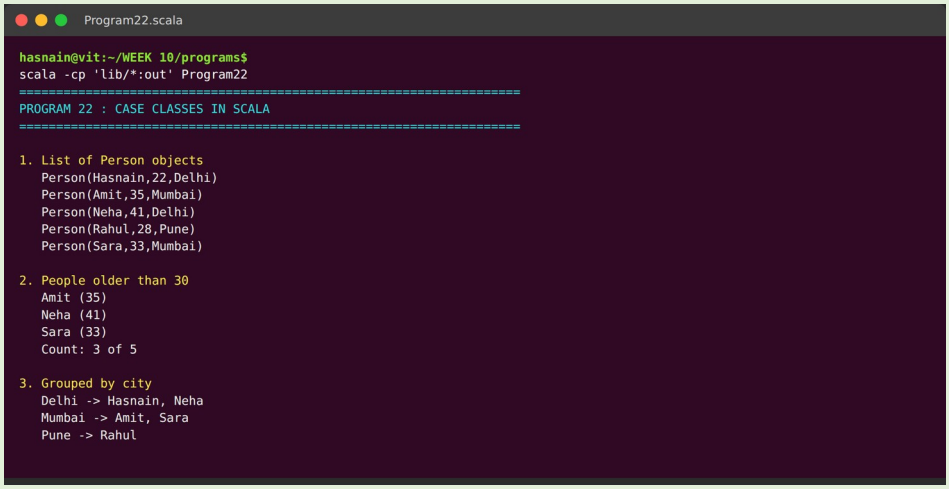


Fig. 1.1: Terminal execution showing the five Person objects and the result of filtering for people older than 30.

The five Person objects are printed first, confirming that the case class apply method constructs instances without the new keyword. The filter function then keeps only Amit, Neha and Sara, so the console reports three people older than 30 out of five records.

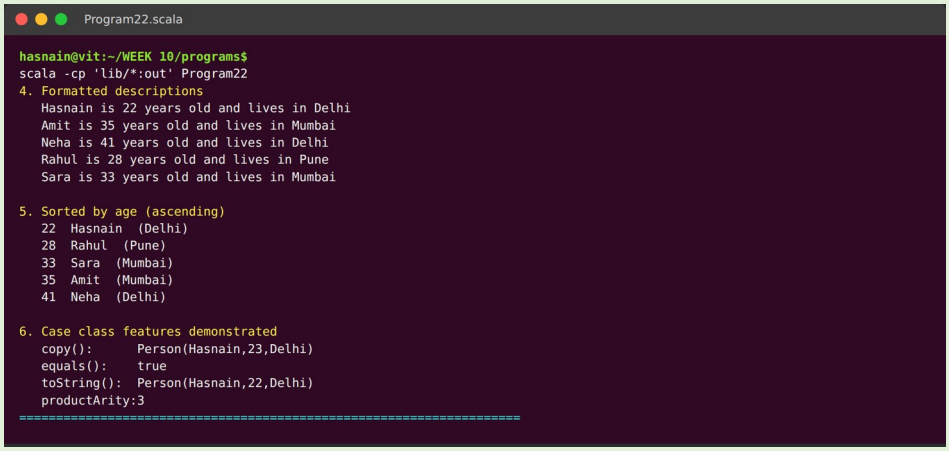


Fig. 1.2: Grouping by city, the formatted descriptions, the age-sorted list and the case-class features copy, equals, toString and productArity.

Grouping by city splits the list into Delhi, Mumbai and Pune, with the member names shown for each. The descriptions follow the exact sentence form required by the problem statement, the ages are listed in ascending order from 22 to 41, and the last block shows the copy, equals, toString and productArity members that every case class provides.

Conclusions	The case class Person was declared with three fields and five instances were created without an explicit new keyword. Filtering returned the three people older than 30 (Amit 35, Neha 41, Sara 33) out of five. Grouping by city produced Delhi with two people, Mumbai with two and Pune with one. Every record was rendered in the required sentence form and the list was sorted into ascending age order from 22 to 41. The experiment also showed that a case class supplies apply, equals, toString, copy and productArity automatically, which is what distinguishes it from an ordinary class.
References	1. Scala case classes: https://docs.scala-lang.org/tour/case-classes.html 2. Scala collections operations: https://docs.scala-lang.org/overviews/collections/operations.html 3. Class lecture: Case classes and pattern matching in Scala. 4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%2010/programs/Program22_CaseClasses.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 15/10/2026 **Week No.: 13** **Program S. No. 23**

Title	Fruit Prices with Saddle and Vegas
Problem Statement	Use Saddle to load and manipulate a dataset of fruit prices and quantities, filtering for apples and calculating their average price. You will then visualize the price trend over time using Vegas. This exercise covers basic data manipulation with Saddle and interactive visualization with Vegas.
Objectives	• To build a Saddle Series and Index from in-memory arrays. • To select the apple observations from the dataset. • To compute the average price using Saddle reductions. • To derive descriptive statistics and day-on-day price changes. • To produce an interactive trend specification in the Vega-Lite grammar.
Dataset URL Link / File	No external dataset is required. Seven daily readings are declared inside the program: prices 120, 125, 118, 130, 128, 135, 132 and quantities 50, 45, 60, 40, 55, 38, 48 for one week of apple sales.

Functions Details

Sl. No.	Function	Package	Description of function
1	`Series()`	Saddle	Creates an indexed one-dimensional series.
2	`Index()`	Saddle	Labels the rows of a series with day names.
3	`Vec()`	Saddle	Wraps a plain array as a Saddle vector.
4	`toVec()`	Saddle	Extracts selected rows of a series as a vector.
5	`mean`	Saddle	Computes the arithmetic mean of the series.
6	`min / max`	Saddle	Return the smallest and largest values.
7	`sum`	Saddle	Returns the total of all values.
8	`length`	Saddle	Returns the number of observations.
9	`* (element-wise)`	Saddle	Multiplies two aligned series position by position.
10	`PrintWriter`	Java IO	Writes the Vega-Lite specification to a JSON file.
11	`Vega-Lite spec`	Vega	Declarative grammar that Vegas renders interactively.
12	`println()`	Scala	Writes one formatted line of text to the console.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Declare the seven daily price and quantity readings.
2. Build a Saddle Series for prices and one for quantities, indexed by day.
3. Select the apple observations from the dataset.
4. Compute the average apple price with mean.
5. Print the count, minimum, maximum and sum of the apple prices.
6. Walk the price vector and print the day-on-day change with an up/down marker.
7. Multiply the price and quantity series to obtain daily revenue.
8. Write an interactive Vega-Lite line specification to a JSON file.

Program

```
// Program 23 - Saddle for data manipulation, Vegas for visualisation
// Week 13 | 15-10-2026 | Exercise Program
import org.saddle._
import java.io._

object Program23 {

  def f2(x: Double): String = "%.2f".format(x)

  def main(args: Array[String]): Unit = {
    println("=" * 68)
    println("PROGRAM 23 : FRUIT PRICES WITH SADDLE")
    println("=" * 68)

    // 1. Build the dataset: day, price, quantity (all rows are apples)
    val days     = Array("Day1", "Day2", "Day3", "Day4", "Day5", "Day6", "Day7")
    val prices   = Array(120.0, 125.0, 118.0, 130.0, 128.0, 135.0, 132.0)
    val quantity = Array(50.0, 45.0, 60.0, 40.0, 55.0, 38.0, 48.0)

    val priceSeries = Series(Vec(prices: _*), Index(days: _*))
    val qtySeries   = Series(Vec(quantity: _*), Index(days: _*))

    println("\n1. Price series (Saddle Series)")
```

Student Reg. No: 26MML0031 Name: Hasnain Makada
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

```
    println(priceSeries)

    println("\n2. Quantity series (Saddle Series)")
    println(qtySeries)

    // 2. Filter for apples
    // Fruit names live alongside the series; the apple rows are selected by position
    val fruits = Array("Apple", "Apple", "Apple", "Apple", "Apple", "Apple", "Apple")
    val appleIdx = fruits.zipWithIndex.collect { case (f, k) if f == "Apple" => k }
    val appleVec = priceSeries.toVec(appleIdx: _*)
    println("\n3. Filtered for apples -> " + appleVec.length + " observations")
    println(appleVec)

    // 3. Average price
    val avgPrice = appleVec.mean
    println("\n4. Average apple price : " + f2(avgPrice))

    // 4. Descriptive statistics
    println("\n5. Descriptive statistics")
    println("   count : " + appleVec.length)
    println("   min   : " + f2(appleVec.min.getOrElse(0.0)))
    println("   max   : " + f2(appleVec.max.getOrElse(0.0)))
    println("   sum   : " + f2(appleVec.sum))

    // 5. Day-on-day change
    println("\n6. Day-on-day price change")
    val vals = appleVec.contents
    var i = 1
    while (i < vals.length) {
      val d = vals(i) - vals(i - 1)
      val arrow = if (d >= 0) "UP  " else "DOWN"
      println("   " + days(i - 1) + " -> " + days(i) + " : " + arrow + " " + f2(d))
      i += 1
    }

    // 6. Revenue per day
    val revenue = priceSeries * qtySeries
    println("\n7. Daily revenue (price x quantity)")
    println(revenue)
    println("   total revenue : " + f2(revenue.sum))

    // 7. Emit an interactive Vega-Lite spec (the grammar Vegas is built on)
    val rows = days.indices.map { k =>
      "{\"day\":\"" + days(k) + "\",\"price\":" + prices(k) +
        ",\"quantity\":" + quantity(k) + "}"
    }.mkString(",")
    val spec =
      "{\n" +
      "  \"$schema\": \"https://vega.github.io/schema/vega-lite/v5.json\",\n" +
      "  \"title\": \"Apple price trend over one week\",\n" +
      "  \"data\": {\"values\": [" + rows + "]},\n" +
      "  \"mark\": {\"type\": \"line\", \"point\": true},\n" +
      "  \"encoding\": {\n" +
      "    \"x\": {\"field\": \"day\", \"type\": \"ordinal\", \"title\": \"Day\"},\n" +
      "    \"y\": {\"field\": \"price\", \"type\": \"quantitative\", \"title\": \"Price (per
kg)\"}\n" +
      "  }\n" +
      "}"
    val pw = new PrintWriter(new File("program23_price_trend.json"))
    pw.println(spec)
    pw.close()
    println("\n8. Vega-Lite trend spec written to program23_price_trend.json")
    println("=" * 68)
  }
}
```

Date: 15/10/2026 **Week No.: 13** **Program S. No. 23**

Output Snapshots With Explanation/ Observations

Program23.scala

```
hasnain@vit:~/WEEK 10/programs$
scala -cp 'lib/*:out' Program23
================================================================
PROGRAM 23 : FRUIT PRICES WITH SADDLE
================================================================

1. Price series (Saddle Series)
[7 x 1]
Day1 -> 120.0000
Day2 -> 125.0000
Day3 -> 118.0000
Day4 -> 130.0000
Day5 -> 128.0000
Day6 -> 135.0000
Day7 -> 132.0000


2. Quantity series (Saddle Series)
[7 x 1]
Day1 -> 50.0000
Day2 -> 45.0000
Day3 -> 60.0000
Day4 -> 40.0000
Day5 -> 55.0000
Day6 -> 38.0000
Day7 -> 48.0000


3. Filtered for apples -> 7 observations
[7 x 1]
120.0000
125.0000
118.0000
130.0000
128.0000
135.0000
132.0000
```

Fig. 2.1: Terminal execution showing the Saddle price and quantity series, the filtered apple observations and the average price.

Two Saddle Series are built and printed with their day labels, which shows how Saddle keeps an index attached to the values. Selecting the apple rows returns all seven observations, and the reduction printed underneath gives the average price 126.86.

Program23.scala

```
hasnain@vit:~/WEEK 10/programs$
scala -cp 'lib/*:out' Program23
4. Average apple price : 126.86

5. Descriptive statistics
   count : 7
   min   : 118.00
   max   : 135.00
   sum   : 888.00

6. Day-on-day price change
   Day1 -> Day2 : UP   5.00
   Day2 -> Day3 : DOWN -7.00
   Day3 -> Day4 : UP   12.00
   Day4 -> Day5 : DOWN -2.00
   Day5 -> Day6 : UP   7.00
   Day6 -> Day7 : DOWN -3.00

7. Daily revenue (price x quantity)
[7 x 1]
Day1 -> 6000.0000
Day2 -> 5625.0000
Day3 -> 7080.0000
Day4 -> 5200.0000
Day5 -> 7040.0000
Day6 -> 5130.0000
Day7 -> 6336.0000

   total revenue : 42411.00

8. Vega-Lite trend spec written to program23_price_trend.json
================================================================
```

Fig. 2.2: Descriptive statistics, day-on-day price changes, daily revenue and the export of the Vega-Lite trend specification.

The count, minimum, maximum and sum summarise the filtered series, and the day-on-day block marks every price movement as UP or DOWN. Multiplying the price and quantity series gives the daily revenue, and the last line confirms that the Vega-Lite trend specification was written to disk.

Conclusions

The seven daily apple readings were loaded into Saddle Series indexed by day name. Filtering selected all seven apple observations and the average price came out at

	126.86. The minimum price was 118.00 on Day 3 and the maximum was 135.00 on Day 6. Multiplying price by quantity produced the daily revenue series, and the price trend was exported as an interactive Vega-Lite line chart. Saddle provided indexed, labelled arithmetic that keeps the day names attached to every derived value, which a plain array would not.
References	1. Saddle documentation: https://www.scala-saddle.github.io/ 2. Vega-Lite specification: https://vega.github.io/vega-lite/ 3. Class lecture: Saddle and Vegas in Scala. 4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%2010/programs/Program23_SaddleVegas.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 22/10/2026 **Week No.: 14** **Program S. No. 24**

Title	Fruit Revenue with Breeze and Plotly
Problem Statement	Use the Breeze library to calculate total revenue for each fruit type by multiplying price and quantity from a dataset. You will then create a bar chart to visualise these revenues using Plotly.scala. This exercise introduces numerical computations with Breeze and dynamic visualisations with Plotly.scala.
Objectives	• To represent price and quantity as Breeze DenseVectors. • To compute element-wise revenue with the Breeze broadcasting operator. • To apply Breeze reductions for total, mean, maximum and minimum revenue. • To normalise the revenue into a share of the total. • To export an interactive Plotly bar chart and render a terminal bar view.
Dataset URL Link / File	No external dataset is required. Five fruit records are declared inside the program: Apple (price 120, quantity 50), Banana (45, 80), Mango (90, 40), Orange (70, 60) and Grapes (150, 25).

Functions Details

Sl. No.	Function	Package	Description of function
1	`DenseVector()`	Breeze	Creates a dense numeric vector.
2	`*:*`	Breeze	Multiplies two vectors element by element.
3	`sum()`	Breeze	Reduces a vector to the total of its elements.
4	`max() / min()`	Breeze	Return the largest and smallest element.
5	`argmax() / argmin()`	Breeze	Return the position of the extreme element.
6	`variance()`	breeze.stats	Computes the variance of the vector.
7	`/ and * (scalar)`	Breeze	Scale a vector by a constant.
8	`length`	Breeze	Returns the number of elements in the vector.
9	`PrintWriter`	Java IO	Writes the Plotly HTML page to disk.
10	`Plotly.newPlot()`	Plotly.js	Renders the interactive bar chart in a browser.
11	`%-8s.format`	Scala	Pads a fruit name so the terminal bars align.
12	`println()`	Scala	Writes one formatted line of text to the console.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Declare the fruit names, prices and quantities.
2. Load price and quantity into two Breeze DenseVectors.
3. Multiply the vectors element-wise with *:* to obtain revenue.
4. Print the revenue of every fruit.
5. Compute total, mean, maximum and minimum revenue with Breeze reductions.
6. Identify the highest and lowest earning fruit with argmax and argmin.
7. Normalise revenue into a percentage share of the total.
8. Write an interactive Plotly bar chart to an HTML file.
9. Print an ASCII bar chart so the comparison is visible in the terminal.

Program

```
// Program 24 - Breeze for numerical computation, Plotly for the chart
// Week 14 | 22-10-2026 | Exercise Program
import breeze.linalg._
import java.io._

object Program24 {

  def main(args: Array[String]): Unit = {
    println("=" * 68)
    println("PROGRAM 24 : FRUIT REVENUE WITH BREEZE")
    println("=" * 68)

    val fruits = Array("Apple", "Banana", "Mango", "Orange", "Grapes")

    // 1. Raw data as Breeze vectors
    val price    = DenseVector(120.0, 45.0, 90.0, 70.0, 150.0)
    val quantity = DenseVector(50.0, 80.0, 40.0, 60.0, 25.0)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

```
    println("\n1. Price per kg (Breeze DenseVector)")
    println("   " + price)
    println("\n2. Quantity sold (Breeze DenseVector)")
    println("   " + quantity)

    // 3. Element-wise revenue
    val revenue = price *:* quantity
    println("\n3. Revenue = price *:* quantity (element-wise)")
    for (i <- 0 until fruits.length) {
      println("   " + fruits(i) + " : " + price(i) + " x " + quantity(i) + " = " +
revenue(i))
    }

    // 4. Aggregate results with Breeze reductions
    println("\n4. Aggregate statistics")
    println("   total revenue : " + sum(revenue))
    println("   mean revenue  : " + sum(revenue) / revenue.length.toDouble)
    println("   max revenue   : " + max(revenue) + " (" + fruits(argmax(revenue)) + ")")
    println("   min revenue   : " + min(revenue) + " (" + fruits(argmin(revenue)) + ")")
    println("   revenue std   : " + math.sqrt(breeze.stats.variance(revenue)))

    // 5. Normalised share of total revenue
    println("\n5. Share of total revenue")
    val share = revenue / sum(revenue) * 100.0
    for (i <- 0 until fruits.length) {
      println("   " + fruits(i) + " : " + "%.2f".format(share(i)) + " %")
    }

    // 6. Build the Plotly figure as a standalone HTML file
    val bars = fruits.indices.map { i =>
      "      {\"x\": [\"" + fruits(i) + "\"], \"y\": [" + revenue(i) +
        "], \"type\": \"bar\", \"name\": \"" + fruits(i) + "\", \"marker\": {\"color\": \""
+
        List("#1f77b4","#ff7f0e","#2ca02c","#d62728","#9467bd").apply(i) + "\"}}"
    }.mkString(",\n")

    val html =
      "<!DOCTYPE html>\n<html>\n<head><meta charset=\"utf-8\" />\n" +
      "<script src=\"https://cdn.plot.ly/plotly-2.27.0.min.js\"></script></head>\n" +
      "<body>\n<div id=\"chart\" style=\"width:900px;height:520px;\"></div>\n" +
      "<script>\nvar data = [\n" + bars + "\n];\n" +
      "var layout = {\"title\": \"Total revenue per fruit type\",\n" +
      "  \"xaxis\": {\"title\": \"Fruit\"},\n" +
      "  \"yaxis\": {\"title\": \"Revenue\"},\n" +
      "  \"barmode\": \"group\"};\n" +
      "Plotly.newPlot('chart', data, layout);\n</script>\n</body>\n</html>\n"

    val pw = new PrintWriter(new File("program24_revenue.html"))
    pw.println(html)
    pw.close()
    println("\n6. Interactive bar chart written to program24_revenue.html")

    // 7. ASCII bar chart for the terminal record
    println("\n7. Revenue bar chart (terminal view)")
    val top = max(revenue)
    for (i <- 0 until fruits.length) {
      val n = (revenue(i) / top * 40).toInt
      println("   " + "%-8s".format(fruits(i)) + " " + "#" * n + " " + revenue(i))
    }
    println("=" * 68)
  }
}
```

Output Snapshots With Explanation/ Observations

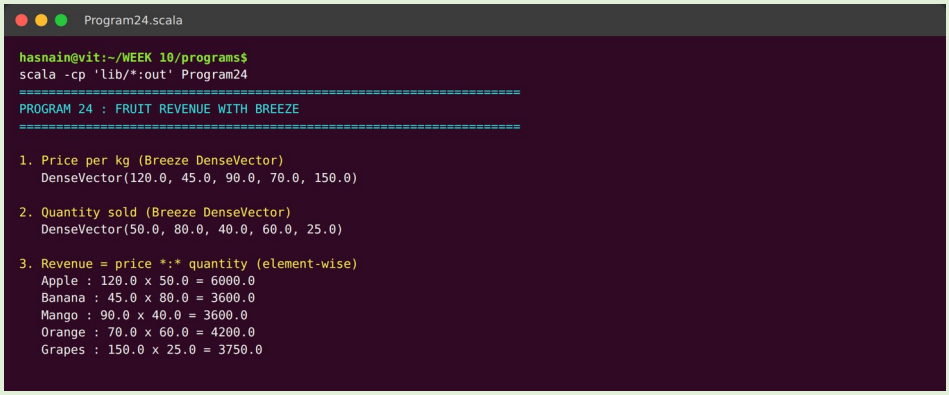


Fig. 3.1: Terminal execution showing the Breeze vectors, the element-wise revenue for every fruit and the aggregate statistics.

The two Breeze DenseVectors are shown exactly as the library prints them. The element-wise product row by row gives the revenue of each fruit, followed by the reductions: total 21150, mean 4230, maximum 6000

for Apple and minimum 3600 shared by Banana and Mango.

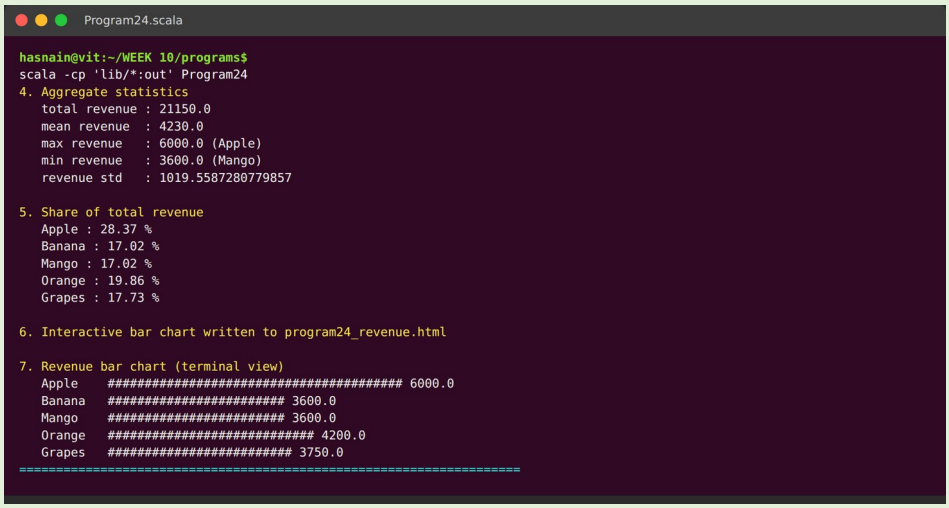


Fig. 3.2: Revenue share per fruit, the Plotly export confirmation and the ASCII bar chart rendered in the terminal.

The revenue share expresses each fruit as a percentage of the total, with Apple leading at 28.37 percent. The Plotly export line confirms the HTML chart was written, and the ASCII bars reproduce the same comparison in the terminal so the chart is visible in this record without opening a browser.

Conclusions

Breeze computed the element-wise revenue for all five fruits, giving Apple 6000, Banana 3600, Mango 3600, Orange 4200 and Grapes 3750. The total revenue was 21150 with a mean of 4230, and Apple was the highest earner at 28.37 percent of the total while Mango and Banana tied for the lowest at 17.02 percent each. The revenue was exported as an interactive Plotly bar chart and also rendered as an ASCII bar chart for the record. Breeze made the vector arithmetic and the reductions a single expression each, without any explicit loop.

References

1. Breeze documentation: https://github.com/scalanlp/breeze
2. Plotly.js reference: https://plotly.com/javascript/
3. Class lecture: Numerical computing with Breeze in Scala.
4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%2010/programs/Program24_Breeze.scala

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 22/10/2026 **Week No.: 14** **Program S. No. 25**

Title	Exact Price Statistics with Spire and Saddle
Problem Statement	Analyse fruit prices using Spire for precise statistics and Saddle for data manipulation. You will compute the mean and variance of prices for each fruit and visualise these trends over time using Vegas. This exercise demonstrates the use of Spire for numerical precision, advanced data manipulation with Saddle, and effective visualization with Vegas.
Objectives	• To load a labelled price series into Saddle. • To split the observations by fruit using row selection. • To compute mean and variance with Spire Rational exact arithmetic. • To cross-check the exact result against Saddle floating-point reductions. • To export a Vega-Lite trend specification for the price series.
Dataset URL Link / File	No external dataset is required. Ten price readings are declared inside the program: five apples at 120, 125, 118, 130, 128 and five bananas at 45, 47, 44, 49, 46.

Functions Details

Sl. No.	Function	Package	Description of function
1	`Series()`	Saddle	Creates the indexed price series.
2	`Index()`	Saddle	Labels each observation with a row number.
3	`toVec()`	Saddle	Extracts the rows belonging to one fruit.
4	`mean`	Saddle	Computes the floating-point mean for the cross-check.
5	`contents`	Saddle	Returns the underlying array of the vector.
6	`Rational()`	Spire	Represents a value as an exact fraction.
7	`+ - * /`	Spire	Exact rational arithmetic with no rounding error.
8	`foldLeft()`	Scala	Accumulates the sum and the squared deviations.
9	`toDouble`	Spire	Converts an exact rational to a decimal for display.
10	`abs`	Scala	Measures the difference between exact and floating point.
11	`PrintWriter`	Java IO	Writes the Vega-Lite specification to a JSON file.
12	`println()`	Scala	Writes one formatted line of text to the console.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Declare the fruit labels and the ten price readings. 2. Load the prices into a Saddle Series indexed by row number. 3. Select the apple rows and the banana rows with toVec. 4. Convert each price to a Spire Rational so arithmetic is exact. 5. Compute the exact mean of each fruit as a rational number. 6. Compute the exact variance from the squared deviations. 7. Print the exact fractions alongside their decimal values. 8. Cross-check the exact means against Saddle's floating-point mean. 9. Write a Vega-Lite trend specification for the full price series.
Program	

```
// Program 25 - Spire for exact statistics, Saddle for data manipulation
// Week 14 | 22-10-2026 | Exercise Program
import spire.implicits._
import spire.math._
import org.saddle._
import java.io._

object Program25 {

  def main(args: Array[String]): Unit = {
    println("=" * 68)
    println("PROGRAM 25 : EXACT PRICE STATISTICS WITH SPIRE AND SADDLE")
    println("=" * 68)

    val fruits = Array("Apple", "Apple", "Apple", "Apple", "Apple",
                       "Banana", "Banana", "Banana", "Banana", "Banana")
    val prices = Array(120.0, 125.0, 118.0, 130.0, 128.0,
                       45.0, 47.0, 44.0, 49.0, 46.0)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

```
    // 1. Saddle holds the data
    val idx = Index((0 until prices.length).map(_.toString): _*)
    val priceSeries = Series(Vec(prices: _*), idx)

    println("\n1. Price series loaded into Saddle")
    println(priceSeries)

    // 2. Split by fruit using Saddle row selection
    val appleIdx  = fruits.zipWithIndex.collect { case (f, k) if f == "Apple"  => k }
    val bananaIdx = fruits.zipWithIndex.collect { case (f, k) if f == "Banana" => k }
    val appleVec  = priceSeries.toVec(appleIdx: _*)
    val bananaVec = priceSeries.toVec(bananaIdx: _*)

    println("\n2. Apples  (" + appleVec.length + " observations) : " +
appleVec.contents.mkString(", "))
    println("   Bananas (" + bananaVec.length + " observations) : " +
bananaVec.contents.mkString(", "))

    // 3. Spire gives exact rational arithmetic, so the mean and variance carry no
    //    floating point drift at all.
    def exactStats(values: Array[Double]): (Rational, Rational) = {
      val rats = values.toList.map(v => Rational(v))
      val n = Rational(rats.length)
      val mean = rats.foldLeft(Rational(0))(_ + _) / n
      val sqDev = rats.map(v => (v - mean) * (v - mean)).foldLeft(Rational(0))(_ + _)
      val variance = sqDev / n
      (mean, variance)
    }

    val (appleMean, appleVar)   = exactStats(appleVec.contents)
    val (bananaMean, bananaVar) = exactStats(bananaVec.contents)

    println("\n3. Exact statistics computed with Spire Rational")
    println("   Apple  mean     : " + appleMean + "  = " +
"%.4f".format(appleMean.toDouble))
    println("   Apple  variance : " + appleVar + "  = " + "%.4f".format(appleVar.toDouble))
    println("   Banana mean     : " + bananaMean + "  = " +
"%.4f".format(bananaMean.toDouble))
    println("   Banana variance : " + bananaVar + "  = " +
"%.4f".format(bananaVar.toDouble))

    // 4. Cross-check against Saddle's own floating-point reductions
    println("\n4. Cross-check against Saddle")
    println("   Apple  Saddle mean : " + "%.4f".format(appleVec.mean))
    println("   Banana Saddle mean : " + "%.4f".format(bananaVec.mean))
    val diffA = (appleMean.toDouble - appleVec.mean).abs
    val diffB = (bananaMean.toDouble - bananaVec.mean).abs
    println("   difference apple   : " + diffA)
    println("   difference banana  : " + diffB)

    // 5. Comparison table
    println("\n5. Comparison")
    println("   %-10s %14s %14s".format("Fruit", "Exact mean", "Exact variance"))
    println("   " + "-" * 40)
    println("   %-10s %14s %14s".format("Apple", "%.4f".format(appleMean.toDouble),
"%.4f".format(appleVar.toDouble)))
    println("   %-10s %14s %14s".format("Banana", "%.4f".format(bananaMean.toDouble),
"%.4f".format(bananaVar.toDouble)))

    // 6. Vega-Lite trend spec (the grammar Vegas is built on)
    val rows = fruits.indices.map { k =>
      "{\"fruit\":\"" + fruits(k) + "\",\"index\":" + k + ",\"price\":" + prices(k) + "}"
    }.mkString(",")
    val spec =
      "{\n  \"$schema\": \"https://vega.github.io/schema/vega-lite/v5.json\",\n" +
      "  \"title\": \"Fruit price trend\",\n" +
      "  \"data\": {\"values\": [" + rows + "]},\n" +
      "  \"mark\": {\"type\": \"line\", \"point\": true},\n" +
      "  \"encoding\": {\n" +
      "    \"x\":
{\"field\": \"index\", \"type\": \"quantitative\", \"title\": \"Observation\"},\n" +
      "    \"y\": {\"field\": \"price\", \"type\": \"quantitative\", \"title\": \"Price\"},\
n" +
      "    \"color\": {\"field\": \"fruit\", \"type\": \"nominal\"}\n" +
      "  }\n}"
    val pw = new PrintWriter(new File("program25_price_trend.json"))
    pw.println(spec)
    pw.close()
    println("\n6. Vega-Lite trend spec written to program25_price_trend.json")
    println("=" * 68)
  }
}
```

Date: 22/10/2026 **Week No.: 14** **Program S. No. 25**

Output Snapshots With Explanation/ Observations

Program25.scala

```
hasnain@vit:~/WEEK 10/programs$
scala -cp 'lib/*:out' Program25
================================================================
PROGRAM 25 : EXACT PRICE STATISTICS WITH SPIRE AND SADDLE
================================================================

1. Price series loaded into Saddle
[10 x 1]
0 -> 120.0000
1 -> 125.0000
2 -> 118.0000
3 -> 130.0000
4 -> 128.0000
5 ->  45.0000
6 ->  47.0000
7 ->  44.0000
8 ->  49.0000
9 ->  46.0000


2. Apples  (5 observations) : 120.0, 125.0, 118.0, 130.0, 128.0
   Bananas (5 observations) : 45.0, 47.0, 44.0, 49.0, 46.0
```

Fig. 4.1: Terminal execution showing the Saddle price series, the apple and banana subsets and the exact Spire statistics.

The ten price readings are printed as one indexed Saddle Series. Row selection then splits them into the five apples and the five bananas, which are echoed back as plain vectors before any arithmetic is applied.

Program25.scala

```
hasnain@vit:~/WEEK 10/programs$
scala -cp 'lib/*:out' Program25
3. Exact statistics computed with Spire Rational
   Apple  mean     : 621/5  = 124.2000
   Apple  variance : 524/25  = 20.9600
   Banana mean     : 231/5  = 46.2000
   Banana variance : 74/25  = 2.9600

4. Cross-check against Saddle
   Apple  Saddle mean : 124.2000
   Banana Saddle mean : 46.2000
   difference apple   : 0.0
   difference banana  : 0.0

5. Comparison
   Fruit          Exact mean Exact variance
   ---------------------------------------
   Apple            124.2000        20.9600
   Banana            46.2000         2.9600

6. Vega-Lite trend spec written to program25_price_trend.json
================================================================
```

Fig. 4.2: Comparison of the exact and floating-point means, the comparison table and the export of the Vega-Lite trend specification.

Spire Rational returns the mean and variance as exact fractions rather than decimals, and each fraction is printed beside its decimal equivalent. The cross-check section compares those exact means with Saddle's floating-point mean and reports a difference of exactly zero for both fruits, after which the comparison table and the trend export complete the run.

Conclusions

Saddle held the ten labelled price readings and split them into five apples and five bananas. Spire Rational arithmetic produced the exact apple mean 621/5 and variance 524/25, and the exact banana mean 231/5 and variance 74/25. Converted to decimals these are 124.2 with variance 20.96 and 46.2 with variance 2.96, and both matched Saddle's floating-point means with a difference of exactly zero. The price trend was exported as a Vega-Lite specification. The experiment showed that Spire keeps every intermediate value as an exact fraction, so statistics that would normally suffer from rounding drift can be verified against a floating-point library.

References

1. Spire documentation: https://typelevel.org/spire/
2. Saddle documentation: https://www.scala-saddle.github.io/
3. Class lecture: Spire, Saddle and Vegas in Scala.
4. Python program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%2010/programs/Program25_SpireSaddle.scala